

Smart Door Lock System

Parth Kothari
23BEC129

Krish Patel
23BEC139

Special Assignment Report

Department of Electronics and Communication Engineering
Institute of Technology, Nirma University
Semester VI — 3EC705CC24-Embedded Systems

Abstract—This report presents the design and implementation of a Smart Door Lock System using the ESP32 microcontroller. The system integrates a 4×4 matrix keypad for physical password entry, a 16x2 LCD display for real-time user feedback, a servo motor as the electromechanical locking actuator, and an active buzzer for audio alerts. Leveraging the ESP32 onboard Wi-Fi capability, the system also hosts a local HTTP web server, allowing users to remotely unlock the door from any browser-equipped device on the same network. Security is reinforced through a three-attempt lockout policy with a 60-second cooldown timer. This project demonstrates practical embedded systems concepts including GPIO interfacing, PWM servo control, I2C communication, matrix keypad scanning, and concurrent web server operation on a resource-constrained microcontroller.

Index Terms—ESP32, Smart Door Lock, IoT, Servo Motor, Keypad, LCD, Web Server, Embedded Systems, Wi-Fi, Access Control, Brute-Force Protection.

I. INTRODUCTION

Traditional mechanical door locks offer limited security and no capability for remote access or monitoring. With the proliferation of microcontroller-based Internet of Things (IoT) platforms, the development of intelligent, connected security systems has become both technically feasible and cost-effective. The ESP32 development board, with its dual-core processor, rich peripheral support, and integrated Wi-Fi and Bluetooth radio, is a particularly capable platform for such embedded IoT applications.

This project implements a dual-interface Smart Door Lock System. Access can be granted either physically via a 4×4 matrix keypad, or remotely through a web browser on any device sharing the same Wi-Fi network. The system provides visual status updates on a 16x2 LCD and auditory feedback via a buzzer, while a servo motor serves as the electromechanical locking actuator. A brute-force protection mechanism locks the system for 60 seconds following three consecutive incorrect password attempts, significantly improving resistance to unauthorized access.

II. LITERATURE SURVEY

Several technologies have been explored in the literature for implementing electronic access control systems:

A. RFID-Based Systems

Radio Frequency Identification (RFID) locks authenticate users via proximity cards or key fobs. While these systems

are convenient, they are susceptible to card cloning attacks and do not inherently support remote access.

B. Biometric Systems

Fingerprint scanners and facial recognition-based locks offer high security by leveraging physiological uniqueness. However, they entail significantly higher hardware costs, computational requirements, and sensitivity to environmental conditions.

C. Bluetooth and Zigbee-Based Systems

Short-range wireless protocols such as Bluetooth Low Energy (BLE) enable proximity-based smartphone unlocking. These systems are limited by range constraints and require a pre-paired device.

D. Wi-Fi and IoT-Based Systems

Wi-Fi-enabled microcontrollers such as the ESP8266 and ESP32 have enabled web-controlled and application-integrated smart locks with potential for global remote access. The ESP32 offers superior performance over ESP8266 through its dual-core processor, additional GPIO pins, and simultaneous Wi-Fi and Bluetooth support, making it the preferred choice for modern IoT embedded applications.

III. LIMITATIONS OF CURRENTLY AVAILABLE TECHNOLOGY

Existing smart lock solutions present several limitations that this project aims to address:

- **High Cost:** Commercial smart lock products are typically priced beyond the reach of budget-constrained deployments such as academic projects and small-scale residential use.
- **Single Authentication Mode:** Many systems rely exclusively on one access method, such as a mobile application or biometric scanner, lacking redundancy in the event of device failure or network unavailability.
- **Cloud Server Dependency:** Several IoT-based lock systems rely on remote cloud infrastructure, making them inoperable during internet outages and introducing data privacy concerns.

- **Absence of Local Feedback:** Minimal implementations often lack real-time local display or audio alert mechanisms, leaving users without contextual feedback during access attempts.
- **No Brute-Force Protection:** Basic keypad locks frequently permit unlimited password attempts, rendering them susceptible to systematic guessing or brute-force attacks.

IV. PROPOSED SOLUTION

The proposed system is a standalone, dual-interface ESP32-based access control solution designed to address the identified limitations:

A. Password Authentication

A 4-digit password is stored in program memory (default: "5678"). The user enters digits via the 4×4 keypad. The "*" key submits the password for comparison, while "*" clears the current input buffer. Entered characters are displayed as asterisks (*) on the LCD to prevent shoulder-surfing.

B. Servo-Based Locking Mechanism

On successful authentication, the servo motor rotates to 90 degrees (unlocked position), holds for 5 seconds to allow door passage, and then automatically returns to 0 degrees (locked position), simulating a physical latch or deadbolt actuator.

C. Brute-Force Lockout Protection

Following three consecutive incorrect password entries, the system activates a 60-second lockout. During this period, all inputs (keypad and web) are rejected. The LCD displays a real-time countdown. A triple-beep auditory alarm activates upon lockout initiation.

D. Wi-Fi Web Interface

Upon boot, the ESP32 connects to a specified Wi-Fi network and hosts an HTTP server on port 80. A mobile-responsive HTML page allows any browser on the network to submit the password. An HTTP 303 (See Other) redirect is implemented to prevent duplicate form submissions on page refresh.

E. Audio-Visual Feedback

The LCD displays contextual status messages ("Enter Password", "Access Granted", "Wrong Password", "System Locked") at each stage. The buzzer produces a single beep on incorrect entry and three rapid beeps upon lockout, providing clear auditory confirmation without requiring visual attention.

V. BLOCK DIAGRAM AND CIRCUIT DESCRIPTION

The system architecture is centred on the ESP32 development board which acts as the central processing unit, managing all peripheral modules and the web server concurrently.

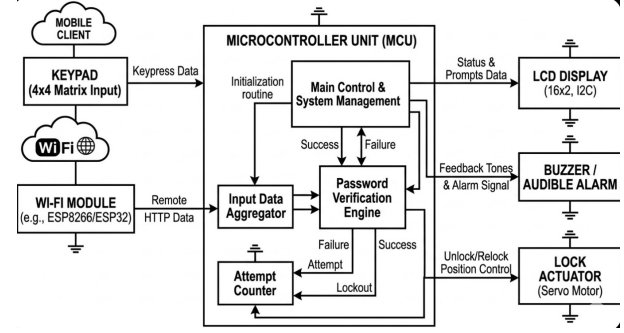


Fig. 1. Block Diagram for the System

TABLE I
HARDWARE COMPONENTS USED

Component	Specification/Details
ESP32 Dev Board	Dual-core Xtensa LX6, 240 MHz, Wi-Fi + Bluetooth integrated
16x2 LCD Display	I2C interface, I2C address 0x27, 16 cols x 2 rows
4x4 Matrix Keypad	Membrane type, 16 keys (0-9, A-D, *, #)
Servo Motor	SG90, PWM-controlled, 0 to 180 degree rotation
Active Buzzer	5V active buzzer for audio feedback
9V Battery	Primary power supply for the entire system
Passive Comps.	Capacitors, Resistors, Jumper Wires

VI. FLOWCHART OF THE CODE

The operational logic of the smart door lock system is illustrated in Fig. 2. The system begins by initializing all hardware peripherals including the LCD display, keypad, servo motor, buzzer, and Wi-Fi module. The ESP32 then starts the HTTP server and displays a prompt requesting the user to enter the password.

The system continuously polls both the keypad and the HTTP client to detect password inputs. If the entered password matches the stored password, the system grants access, activates the buzzer briefly, and rotates the servo motor to the unlocked position. After a delay of five seconds, the servo returns to the locked position and the wrong attempt counter is reset.

If the password is incorrect, the system displays an access denied message and increments the wrong attempt counter. When three incorrect attempts occur consecutively, the system

TABLE II
ESP32 PIN CONFIGURATION

Module	Interface	ESP32 Pins
Keypad	Digital In	Rows: 32, 33, 25, 26 Cols: 27, 14, 12, 13
16x2 LCD	I2C	SDA: 21, SCL: 22
Servo Motor	PWM Out	GPIO 18
Active Buzzer	Digital Out	GPIO 19
Wi-Fi	RF Module	Internal

enters a lockout state for 60 seconds during which all inputs are ignored.

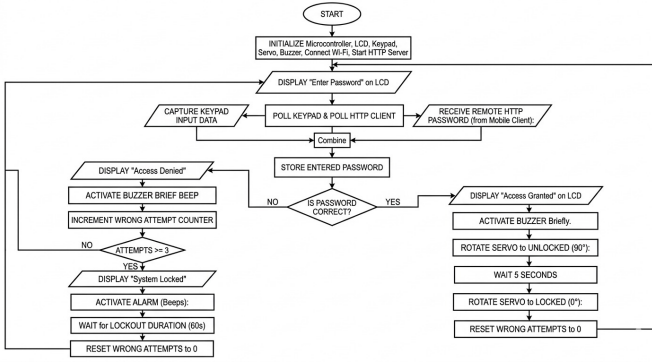


Fig. 2. Operational flowchart of the ESP32-based smart door lock system.

VII. RESULTS

A. Hardware Implementation Results

The hardware was assembled on a general-purpose PCB using the ESP32 as the central controller. All peripheral modules were verified individually before system integration, and functionalities were demonstrated successfully.

TABLE III
TEST CASES AND OBSERVED RESULTS

Test Case	Input Action	Expected Output
Correct Pass	'5678' + '#'	Servo 90 deg, LCD: Access Granted (Pass)
Wrong Pass	'1234' + '#'	Beep, LCD: Wrong Password (Pass)
3x Wrong	3x incorrect	Triple beep, System locked (Pass)
Lockout Display	During 60s	LCD: System Locked — Try after Xs (Pass)
Clear Input	Press '*'	Buffer cleared, LCD reset (Pass)
Web Unlock	Submit '5678'	Servo unlocks, HTTP 303 (Pass)
Web Lockout	3x wrong web	System locks (Pass)

B. Web Interface Observation

On power-up, the ESP32 connected to the specified Wi-Fi network and displayed its assigned local IP address on the LCD. The hosted web page loaded correctly on both smartphone and desktop browsers, presenting a clean gradient-styled form. Form submissions triggered the /unlock route, processed the authentication, and issued an HTTP 303 redirect.

VIII. ANALYSIS OF RESULT

The experimental evaluation confirms that the ESP32 platform is capable of handling both hardware interfacing and lightweight web server functionality simultaneously. The keypad input processing, LCD feedback, and servo control operations are executed with minimal latency, ensuring a responsive user experience.

TABLE IV
SYSTEM PERFORMANCE OBSERVATIONS

Performance Parameter	Observed Value
Wi-Fi Connection Time	3-5 seconds
Keypad Response Latency	<50 ms
Servo Actuation (0 to 90 deg)	~300-400 ms
LCD Update Speed	<10 ms
Web Server Response Time	~200-400 ms

The Wi-Fi based interface introduces an additional layer of convenience by enabling remote door unlocking through any browser-enabled device on the local network. Unlike many IoT implementations that depend on cloud infrastructure, the system operates entirely on a local HTTP server hosted by the ESP32, thereby improving privacy and reducing network dependency.

The brute-force protection mechanism proved effective during testing. After three consecutive incorrect password attempts, the system entered a lockout state and ignored further inputs until the countdown timer expired. This behavior significantly reduces the possibility of unauthorized access attempts through repeated guessing.

Overall, the prototype demonstrates that a secure and low-cost smart lock system can be implemented using widely available embedded components without requiring complex infrastructure.

IX. ADVANTAGES OF THE PROPOSED SYSTEM

The proposed ESP32-based smart door lock system offers several advantages compared to traditional locking mechanisms and basic electronic locks:

- **Dual Access Modes:** The system supports both keypad-based authentication and web-based remote access, providing flexibility and redundancy.
- **Low Cost Implementation:** All hardware components used in the system are inexpensive and widely available, making the design suitable for educational and residential applications.
- **Standalone Operation:** The system operates independently without requiring cloud servers, improving reliability and protecting user privacy.
- **Enhanced Security:** The brute-force lockout mechanism prevents repeated password attempts, significantly reducing the risk of unauthorized access.
- **Real-Time Feedback:** The LCD display and buzzer provide immediate audio-visual feedback during authentication attempts.
- **Energy Efficient:** The ESP32 microcontroller offers efficient power consumption while providing powerful processing and networking capabilities.
- **Scalable Architecture:** The design can be easily extended with additional features such as biometric authentication, mobile applications, or cloud integration.

X. APPLICATIONS

The developed smart door lock system can be applied in several real-world scenarios where secure and convenient access control is required.

- **Residential Security:** Smart locks can be used in homes and apartments to provide secure access without traditional mechanical keys.
- **Office Access Control:** Small offices and laboratories can deploy such systems to restrict entry to authorized personnel.
- **Hostel and Dormitory Rooms:** Educational institutions can use electronic locks to improve security and manage access more effectively.
- **Storage Rooms and Cabinets:** The system can be adapted for secure access to lockers, cabinets, or restricted storage areas.
- **IoT-Based Home Automation:** The lock can be integrated into larger smart home ecosystems to enable remote monitoring and automated security operations.

XI. FUTURE SCOPE

Although the proposed system successfully demonstrates a functional smart door lock prototype, several enhancements can be implemented in future versions to further improve functionality, security, and usability.

- **Non-Volatile Password Storage:** The password can be stored in ESP32 Non-Volatile Storage (NVS) memory, allowing users to change the password dynamically without modifying the program.
- **Mobile Application Integration:** A dedicated Android or iOS application can be developed to provide a more user-friendly remote control interface.
- **Biometric Authentication:** Integration of fingerprint sensors or facial recognition modules can significantly enhance the security level of the system.
- **Cloud Connectivity:** The system can be connected to IoT platforms such as MQTT brokers or cloud dashboards to enable remote monitoring and logging of access events.
- **Camera-Based Monitoring:** A small camera module could capture images during access attempts for additional security verification.
- **Battery Backup System:** Incorporating rechargeable battery backup and power management circuits would ensure uninterrupted operation during power failures.

XII. CONCLUSION

The ESP32-based Smart Door Lock System was successfully designed and implemented as a low-cost embedded security solution. The system integrates a matrix keypad, LCD display, servo motor actuator, and Wi-Fi-based web interface to provide a flexible dual-mode authentication mechanism. The project demonstrates the practical application of embedded systems concepts including GPIO interfacing, PWM motor control, I2C communication, keypad scanning, and HTTP server implementation.

The system performed reliably during testing, providing fast response times and accurate authentication handling. The three-attempt lockout feature significantly improves security by preventing brute-force password attempts. Additionally, the integrated web interface enhances convenience by allowing users to unlock the door remotely using a browser.

Overall, the developed prototype highlights the potential of ESP32-based embedded platforms for implementing secure and intelligent IoT-enabled access control systems. With further enhancements such as biometric authentication, cloud connectivity, and mobile application support, the system can be extended into a fully featured commercial smart lock solution.

REFERENCES

- [1] S. Bhosale, R. Patil, and A. Shinde, "RFID Based Door Locking System," *Int. Journal of Engineering Research and Technology (IJERT)*, vol. 3, no. 3, pp. 1234-1237, 2014.
- [2] M. A. Saleem and Z. Iqbal, "Smart Door Lock System using Fingerprint Sensor," *Proc. IEEE Int. Conference on Computing, Mathematics and Engineering Technologies (iCoMET)*, 2019.
- [3] A. Kumar and B. Tripathi, "Bluetooth-Based Home Automation and Security System using Arduino," *Int. Journal of Computer Applications*, vol. 131, no. 7, pp. 20-25, 2015.
- [4] R. Patel and S. Mehta, "IoT-Based Smart Door Lock using ESP8266 and Android Application," *Int. Journal of Innovative Research in Science, Engineering and Technology*, vol. 7, no. 5, 2018.
- [5] Espressif Systems, "ESP32 Technical Reference Manual," Version 5.0, 2023.
- [6] M. Stanley, "ESP32Servo Arduino Library," GitHub, 2022.
- [7] F. de Brabander, "LiquidCrystal_I2C Arduino Library," GitHub, 2019.

APPENDIX: SOURCE CODE

```

1 #include <WiFi.h>
2 #include <WebServer.h>
3 #include <LiquidCrystal_I2C.h>
4 #include <Keypad.h>
5 #include <ESP32Servo.h>
6
7 const char* ssid = "Hehe";
8 const char* password = "12345679";
9
10 WebServer server(80);
11
12 LiquidCrystal_I2C lcd(0x27,16,2);
13
14 Servo myServo;
15 int servoPin = 18;
16
17 int buzzerPin = 19;
18
19 String correctPassword = "5678";
20 String enteredPassword="";
21
22 int wrongAttempts=0;
23 bool locked=false;
24 unsigned long lockStartTime=0;
25 const int lockDuration=60;
26
27 // Keypad
28 const byte ROWS=4;
29 const byte COLS=4;
30
31 char keys[ROWS][COLS]={
32 {'1','2','3','A'},
33 {'4','5','6','B'},
34 {'7','8','9','C'},
35 {'*','0','#','D'}
36 };
37
38 byte rowPins[ROWS]={32,33,25,26};
39 byte colPins[COLS]={27,14,12,13};
40
41 Keypad keypad=Keypad(makeKeymap(keys),rowPins,colPins,ROWS,
42 COLS);
43
44 void beep()
45 {
46     digitalWrite(buzzerPin, HIGH);
47     delay(300);
48     digitalWrite(buzzerPin, LOW);
49 }
50
51 void checkPassword(String input)
52 {
53     if(locked) return;
54
55     lcd.clear();
56
57     if(input==correctPassword)
58     {
59         lcd.print("Access_Granted");
60
61         wrongAttempts=0;
62
63         myServo.write(90);
64         delay(5000);
65         myServo.write(0);
66     }
67     else
68     {
69         lcd.print("Wrong_Password");
70
71         beep(); // BUZZER SOUND
72
73         wrongAttempts++;
74
75         if(wrongAttempts>=3)
76         {
77             locked=true;
78             lockStartTime=millis();
79
80             // alarm beep
81             for(int i=0;i<3;i++)
82
83         {
84             beep();
85             delay(200);
86         }
87
88         lcd.clear();
89
90         delay(2000);
91
92         if(!locked)
93         {
94             lcd.clear();
95             lcd.print("Enter_Password");
96         }
97     }
98
99 void handleRoot()
100 {
101     String page = R"rawliteral(
102     <!DOCTYPE_html>
103     <html>
104     <head>
105     <title>Smart_Door_Lock</title>
106     <meta_name="viewport" _content="width=device-width, initial-
107         scale=1">
108     <style>
109     body{
110         _font-family:_Arial;
111         _background:_linear-gradient(135deg,#4facfe,#00f2fe);
112         _text-align:center;
113         _color:white;
114     }
115
116     .container{
117         _margin-top:60px;
118     }
119
120     h2{
121         _font-size:30px;
122     }
123
124     input[type=password]{
125         _padding:12px;
126         _width:200px;
127         _font-size:18px;
128         _border:none;
129         _border-radius:10px;
130         _text-align:center;
131     }
132
133     button{
134         _padding:12px_25px;
135         _margin-top:15px;
136         _font-size:18px;
137         _border:none;
138         _border-radius:10px;
139         _background:#28a745;
140         _color:white;
141         _cursor:pointer;
142     }
143
144     button:hover{
145         _background:#218838;
146     }
147
148     .card{
149         _background:rgba(255,255,255,0.2);
150         _padding:30px;
151         _border-radius:20px;
152         _width:300px;
153         _margin:auto;
154         _box-shadow:0px_10px_25px_rgba(0,0,0,0.3);
155     }
156     </style>
157     </head>
158
159     <body>
160
161     <div_class="container">
162     <div_class="card">

```

```

163 <h2>_Smart_Door_Lock</h2>
164
165 <form_action="/unlock">
166 <p>Enter_Password</p>
167
168 <input_type="password"_name="pass"><br>
169
170 <button_type="submit">Unlock_Door</button>
171
172 </form>
173
174 </div>
175 </div>
176
177 </body>
178 </html>
179 )rawliteral";
180
181 server.send(200, "text/html", page);
182 }
183
184 void handleUnlock()
185 {
186   String pass=server.arg("pass");
187
188   checkPassword(pass);
189
190   server.sendHeader("Location", "/");
191   server.send(303);
192 }
193
194 void setup()
195 {
196   Serial.begin(115200);
197
198   lcd.init();
199   lcd.backlight();
200
201   pinMode(buzzerPin, OUTPUT);
202
203   lcd.print("Connecting_WiFi");
204
205   Serial.println("Connecting_WiFi...");
206
207   WiFi.begin(ssid, password);
208
209   while(WiFi.status() != WL_CONNECTED)
210   {
211     delay(500);
212     Serial.print(".");
213   }
214
215   Serial.println("");
216   Serial.println("WiFi_connected");
217   Serial.print("IP_Address:");
218   Serial.println(WiFi.localIP());
219
220   lcd.clear();
221   lcd.print("WiFi_Connected");
222
223   delay(2000);
224
225   lcd.clear();
226   lcd.print("Enter_Password");
227
228   server.on("/", handleRoot);
229   server.on("/unlock", handleUnlock);
230
231   server.begin();
232
233   myServo.attach(servoPin);
234   myServo.write(0);
235 }
236
237 void loop()
238 {
239   server.handleClient();
240
241   if(locked)
242   {
243     int elapsed=(millis()-lockStartTime)/1000;
244     int remaining=lockDuration-elapsed;
245

```

```

246   if(remaining<=0)
247   {
248     locked=false;
249     wrongAttempts=0;
250
251     lcd.clear();
252     lcd.print("Enter_Password");
253     return;
254   }
255
256   lcd.setCursor(0,0);
257   lcd.print("System_Locked");
258
259   lcd.setCursor(0,1);
260   lcd.print("Try_after_");
261   lcd.print(remaining);
262   lcd.print("_");
263
264   delay(1000);
265   return;
266 }
267
268 char key=keypad.getKey();
269
270 if(key)
271 {
272   if(key=='#')
273   {
274     checkPassword(enteredPassword);
275     enteredPassword="";
276   }
277   else if(key=='*')
278   {
279     enteredPassword="";
280     lcd.clear();
281     lcd.print("Enter_Password");
282   }
283   else
284   {
285     if(enteredPassword.length()<4)
286     {
287       enteredPassword+=key;
288
289       lcd.setCursor(enteredPassword.length()-1,1);
290       lcd.print("*");
291     }
292   }
293 }
294 }

```